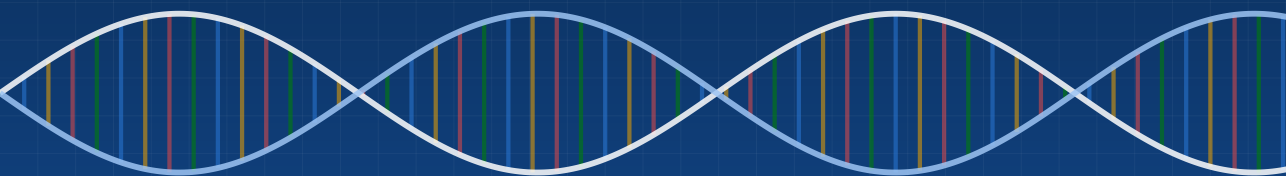


Mapeo de lecturas



Ignorar sin más las repeticiones no es una opción: eso crea sus propios problemas y puede hacer que pasen inadvertidos fenómenos biológicos importantes.

— Treangen y Salzberg (2012)

● Al terminar este capítulo usted podrá

- Explicar por qué el alineamiento por programación dinámica no escala al mapeo de cientos de millones de lecturas, y comparar las dos familias de índices que lo resuelven: tablas de *hash* de semillas e índices de sufijos.
- Construir a mano el arreglo de sufijos, la transformada de Burrows-Wheeler y el índice FM de una secuencia corta, y ejecutar paso a paso la búsqueda hacia atrás con el *LF-mapping*.
- Describir el paradigma semilla-encadenamiento-extensión de BWA-MEM y minimap2, incluidos los minimizadores, la función de encadenamiento y la calidad de mapeo (MAPQ), y leer sin ayuda un registro SAM.
- Construir un flujo reproducible con `samtools` (ordenar, marcar duplicados, indexar), inspeccionar alineamientos en IGV y usar perfiles de cobertura para detectar deleciones y duplicaciones.

Al terminar el ?? teníamos en las manos un archivo FASTQ limpio: millones de fragmentos de 150 bases, cada uno con su calidad Phred, y una teoría (la de Lander-Waterman) que nos decía cuántos necesitábamos para cubrir un genoma. Pero una lectura suelta no significa nada; solo adquiere sentido cuando sabemos *de dónde* del genoma procede. Esa es la tarea del *mapeo*: colocar cada lectura en su lugar de origen dentro de una secuencia de referencia. Casi todo el análisis genómico moderno (el llamado de variantes del próximo capítulo, la cuantificación de expresión con RNA-seq, los picos de ChIP-seq, la metilación) empieza con este paso, y hereda sus errores.

El problema parece una aplicación directa del ??: basta con alinear cada lectura contra el genoma con Smith-Waterman. La dificultad es de escala. Un genoma humano a 30× son unos 620 millones de lecturas de 150 pares de bases, y la referencia mide $3,1 \times 10^9$ bases. Li y Homer (2010) resumieron la respuesta de la comunidad en una frase que hoy parece obvia pero que en 2008 no lo era: con los algoritmos adecuados, el alineamiento de lecturas cortas “ha dejado de ser el cuello de botella” del análisis. Este capítulo cuenta cómo se logró.

Empezaremos por la idea algorítmica más bella del área: una permutación de la secuencia, la transformada de Burrows-Wheeler, que permite buscar cualquier patrón en un genoma humano ocupando menos memoria que el propio genoma (sección 7.1). Después veremos cómo las herramientas reales (BWA, Bowtie, minimap2) combinan índices, semillas y programación dinámica, cómo expresan su confianza con la calidad de mapeo y cómo escriben el resultado en el formato SAM/BAM que manipularemos con [samtools](#) (sección 7.2). Terminaremos mirando los alineamientos con los ojos, en IGV, y convirtiéndolos en perfiles de cobertura capaces de revelar deleciones y duplicaciones (sección 7.3).

7.1 Transformada de Burrows-Wheeler y FM-index

 [Colab](#) · 7.1 · Transformada de Burrows-Wheeler y FM-index  [repositorio del curso](#)

Un diccionario de todos los finales

Imagine un diccionario muy extraño: en lugar de palabras, contiene *todos los finales* de un libro. Cada entrada empieza en una letra distinta del texto

y continúa hasta el punto final, y todas están ordenadas alfabéticamente. Si quiere saber dónde aparece la frase “había una vez”, no necesita leer el libro: abre el diccionario por la mitad, decide si su frase va antes o después, vuelve a partir por la mitad, y en una veintena de saltos llega al bloque de entradas que empiezan con esa frase. Cada entrada, además, recuerda en qué página empezaba. El problema es que el diccionario pesa muchísimo más que el libro. La transformada de Burrows-Wheeler es el truco que permite quedarse con una sola columna de ese diccionario, tan delgada como el propio texto, sin perder la capacidad de buscar.

7.1.1 El problema del mapeo

Formalicemos lo que pedimos a un mapeador. Sea $R = r_1 r_2 \dots r_G$ la secuencia de referencia (un genoma, con sus cromosomas concatenados) y $q = q_1 \dots q_m$ una lectura, con $m \ll G$.

Definición 7.1. (Mapeo de una lectura)

Dada una función de puntuación de alineamiento local $S(q, R_{u..v})$ (por ejemplo, la de Smith-Waterman con huecos afines del ??), mapear q consiste en encontrar el intervalo de la referencia

$$(\hat{u}, \hat{v}) = \arg \max_{1 \leq u \leq v \leq G} \max \{S(q, R_{u..v}), S(\bar{q}, R_{u..v})\}, \quad (7.1)$$

y en cuantificar la probabilidad de que $(\hat{u}, \hat{v})$ sea el origen verdadero de la lectura.

DONDE

R, G	Secuencia de referencia y su longitud en bases.
q, m	Lectura y su longitud.
$\bar{q}$	Reverso complementario de la lectura: el fragmento pudo secuenciarse desde cualquiera de las dos hebras.
$R_{u..v}$	Subcadena de la referencia entre las posiciones u y v .
$S(\cdot, \cdot)$	Puntuación del mejor alineamiento local entre la lectura y la subcadena.

La segunda mitad de la definición es tan importante como la primera, y a menudo se olvida: un mapeador no solo responde *dónde*, sino *con qué seguridad*. Volveremos a ello con la calidad de mapeo en la sección 7.2.

Resolver la ecuación (7.1) por fuerza bruta es imposible. Con Smith-Waterman, cada lectura exige llenar una matriz de $m \times G = 150 \times 3,1 \times 10^9 \approx 4,7 \times 10^{11}$ celdas; multiplicado por $6,2 \times 10^8$ lecturas, son unos $2,9 \times 10^{20}$ celdas. Aun a mil millones de celdas por segundo, el cálculo tardaría unos nueve mil años. Necesitamos *filtrar*: descartar en un instante casi todo el genoma y reservar la programación dinámica para unas pocas regiones candidatas.

7.1.2 Semillas y tablas de hash

La primera generación de mapeadores tomó prestada la idea de BLAST: si la lectura procede de la posición u , casi seguro comparte con la referencia algún k -mero exacto (una *semilla*). Basta con un índice que, para cada k -mero, liste las posiciones donde aparece. ¿Qué k elegir? Si el genoma fuera una secuencia aleatoria de bases equiprobables, el número esperado de apariciones *casuales* de un k -mero concreto sería

$$E[N_k] = \frac{G - k + 1}{4^k} \approx \frac{G}{4^k}. \tag{7.2}$$

DONDE	
N_k	Número de posiciones de la referencia donde aparece, por azar, un k -mero dado.
$G - k + 1$	Número de k -meros (posiciones de inicio) en la referencia.
4^k	Número de k -meros distintos posibles sobre {A, C, G, T}.

Para el genoma humano, un 12-mero aparece por azar unas 185 veces, un 16-mero 0,72 veces y un 20-mero apenas 0,0028 veces (figura 7.1). Las semillas deben tener, pues, unas 20 bases para ser *informativas*. Pero una tabla directa indexada por k -mero necesita 4^k entradas: con enteros de 32 bits, 64 MiB para $k = 12$ y 16 GiB para $k = 16$, antes de guardar una sola posición. Por eso los mapeadores basados en *hash*, como MAQ (Li et al., 2008), indexaban semillas cortas, a menudo espaciadas, y a veces indexaban las *lecturas* en lugar del genoma. Funcionaban, pero eran lentos cuando había que escalar a cientos de individuos (Li & Durbin, 2009).

La alternativa, que acabó por imponerse para lecturas cortas, fue indexar *todos los sufijos* de la referencia. Un índice de sufijos no depende de un k fijo: permite preguntar por un patrón de cualquier longitud y, como veremos, extenderlo carácter a carácter hasta que deje de aparecer.

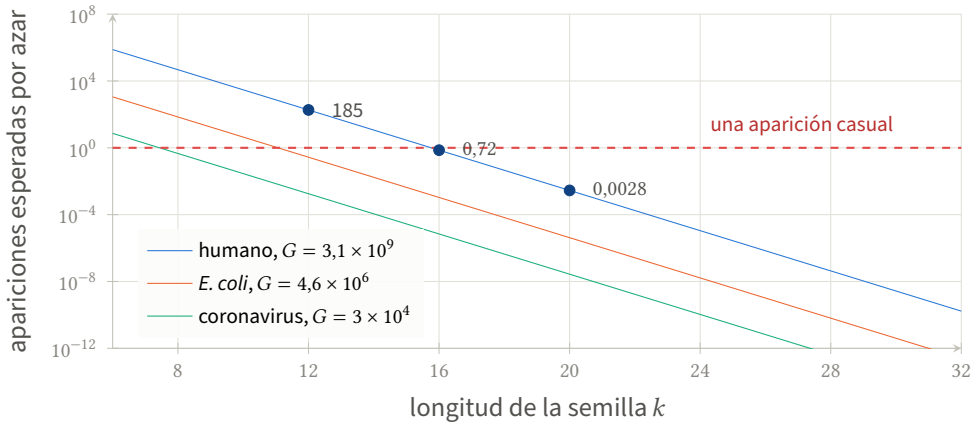


Figura 7.1. Número esperado de apariciones casuales de un k -mero en genomas aleatorios de distinto tamaño (ecuación (7.2)). En escala logarítmica cada base adicional divide la cifra por cuatro. En el genoma humano, una semilla necesita unas 20 bases para que una coincidencia exacta sea más probablemente verdadera que casual; en un virus basta con 8 o 9. Los genomas reales no son aleatorios: sus repeticiones hacen que muchos k -meros largos aparezcan miles de veces, como veremos en la sección 7.2.

7.1.3 El arreglo de sufijos

Añadimos al final de la referencia un símbolo centinela $\$$ que no pertenece al alfabeto y que, por convenio, es menor que cualquier base. Así, $T = R\$$ tiene longitud $n = G + 1$ y ningún sufijo es prefijo de otro.

Definición 7.2. (Arreglo de sufijos)

El arreglo de sufijos de T es la permutación $SA[0..n-1]$ de $\{0, \dots, n-1\}$ tal que

$$T[SA[0]..] < T[SA[1]..] < \dots < T[SA[n-1]..] \quad (7.3)$$

en orden lexicográfico, donde $T[i..]$ denota el sufijo que empieza en la posición i (numeramos desde cero).

DONDE

T, n	Texto indexado (referencia más centinela) y su longitud.
$T[i..]$	Sufijo de T que empieza en la posición i .
$SA[r]$	Posición en T del sufijo que ocupa el lugar r en el orden alfabético.

La propiedad clave es que todos los sufijos que empiezan por un patrón P quedan *contiguos* en el arreglo, porque comparten prefijo. Encontrar las apariciones de P se reduce a encontrar un *intervalo* $[sp, ep)$ de filas, lo que se hace con dos búsquedas binarias en $O(m \log n)$ comparaciones de caracteres. Manber y Myers (1993) propusieron esta estructura como alternativa compacta a los árboles de sufijos, que resuelven el mismo problema en tiempo $O(m)$ pero ocupan, en la práctica, más de diez bytes por base (Gusfield, 1997). Aun así, el arreglo de sufijos necesita $n \lceil \log_2 n \rceil$ bits: para el genoma humano, 32 bits por posición, unos 12 GB. En 2008, eso era más memoria de la que tenía un servidor típico.

7.1.4 La transformada de Burrows-Wheeler

Escribamos todas las rotaciones cíclicas de T , una debajo de otra, y ordenémoslas alfabéticamente. Obtenemos una matriz M de $n \times n$ caracteres (figura 7.2). Como T termina en el centinela único, ordenar las rotaciones equivale a ordenar los sufijos: la fila r de M empieza con el sufijo $T[SA[r]..]$.

Definición 7.3. (Transformada de Burrows-Wheeler)

La BWT de T es la última columna L de la matriz de rotaciones ordenadas. Equivalentemente, a partir del arreglo de sufijos,

$$L[r] = \begin{cases} T[SA[r] - 1] & \text{si } SA[r] > 0, \\ \$ & \text{si } SA[r] = 0. \end{cases} \quad (7.4)$$

Llamamos F a la primera columna de M : son los caracteres de T ordenados.

DONDE

M	Matriz cuyas filas son las n rotaciones de T en orden lexicográfico.
L	Última columna de M : la transformada de Burrows-Wheeler, BWT(T).
F	Primera columna de M ; se deduce de L ordenando sus caracteres.

Rotaciones de T		Matriz ordenada M	
i		$r(SA)$	
0	T A C A T A C A G \$	0 (9)	\$ T A C A T A C A G
1	A C A T A C A G \$ T	1 (5)	A C A G \$ T A C A T
2	C A T A C A G \$ T A	2 (1)	A C A T A C A G \$ T
3	A T A C A G \$ T A C	3 (7)	A G \$ T A C A T A C
4	T A C A G \$ T A C A	4 (3)	A T A C A G \$ T A C
5	A C A G \$ T A C A T	5 (6)	C A G \$ T A C A T A
6	C A G \$ T A C A T A	6 (2)	C A T A C A G \$ T A
7	A G \$ T A C A T A C	7 (8)	G \$ T A C A T A C A
8	G \$ T A C A T A C A	8 (4)	T A C A G \$ T A C A
9	\$ T A C A T A C A G	9 (0)	T A C A T A C A G \$
			F L

ordenar →

$BWT(T) = L = GTTCCAAAA\$$

Figura 7.2. Construcción de la BWT de $T = TACATACAG\$$. **Izquierda:** las diez rotaciones cíclicas de T , indexadas por su posición de inicio i . **Derecha:** las mismas rotaciones en orden alfabético ($\$$ precede a todas las bases). Junto a cada fila r aparece, entre paréntesis, $SA[r]$. La primera columna F (azul) contiene las letras de T ordenadas; la última, L (naranja), es la transformada. Observe cómo las cuatro A de L quedan juntas: los sufijos que empiezan por C, G o T tienden a estar precedidos por la misma letra, y la BWT agrupa esas repeticiones. Generada con `figuras/cap07/generar.py`.

En palabras: $L[r]$ es el carácter que *precede*, en el texto, al sufijo de la fila r . Para nuestro ejemplo de juguete, $T = TACATACAG\$$, el arreglo de sufijos es (9, 5, 1, 7, 3, 6, 2, 8, 4, 0) y la transformada es $L = GTTCCAAAA\$$.

🔗 Un algoritmo de compresión que nadie esperaba

Michael Burrows y David Wheeler describieron la transformada en 1994, en un reporte técnico del Centro de Investigación de Sistemas de Digital Equipment Corporation, con un propósito muy distinto al nuestro: comprimir texto. Su observación fue que L tiende a formar rachas largas de un mismo carácter (en inglés, las letras que preceden a “he” suelen ser t), lo que la hace muy fácil de comprimir, y que, sorprendentemente, la permutación es *reversible*. El compresor `bzip2` se basa en ella. Adjeroh et al. (2008) reconstruyen esta historia y su conexión con los arreglos de sufijos. Pasaron seis años hasta que Paolo Ferragina y Giovanni Manzini, en un congreso de 2000, se dieron cuenta de que la BWT comprimida no solo guardaba el texto, sino que permitía *buscar* en él; la versión extendida de su trabajo es Ferragina

y Manzini (2005). Y pasaron otros nueve años hasta que Bowtie y BWA llevaron la idea a la genómica (Langmead et al., 2009; Li & Durbin, 2009).

7.1.5 El *LF-mapping*: por qué la BWT es reversible

¿Cómo recuperar T a partir de L sola? La clave es una propiedad que, una vez vista, parece evidente.

Teorema 7.1. *(Orden de las apariciones)*

La j -ésima aparición de un carácter c en la columna L y la j -ésima aparición de c en la columna F corresponden a la misma posición del texto T .

Demostración. Considere dos filas de M cuya última letra es c , digamos cX rotada como Xc y Yc , con $X < Y$ porque las filas están ordenadas. Si rotamos ambas un carácter a la derecha obtenemos cX y cY , que son también filas de M (con c en la columna F), y como comparten el primer carácter, su orden relativo lo decide el resto: $cX < cY$. El orden entre las c se conserva al pasar de L a F . $\square$

Esta propiedad permite definir, para cada fila r , la fila $LF(r)$ en la que aparece como primera letra el mismo carácter que $L[r]$. Necesitamos dos tablas:

$$C[c] = \#\{i : T[i] < c\}, \quad (7.5)$$

$$\text{Occ}(c, r) = \#\{i < r : L[i] = c\}, \quad (7.6)$$

y con ellas

$$LF(r) = C[L[r]] + \text{Occ}(L[r], r). \quad (7.7)$$

DONDE

$C[c]$	Número de caracteres de T estrictamente menores que c : la fila donde empieza el bloque de c en la columna F .
$\text{Occ}(c, r)$	Número de apariciones de c en las primeras r filas de L (filas 0 a $r - 1$): el rango de $L[r]$ entre sus iguales.
$LF(r)$	Fila de M cuyo primer carácter es el mismo carácter del texto que $L[r]$; equivale a retroceder una posición en T .

La ecuación (7.7) es la traducción directa del teorema: el bloque de c en F empieza en la fila $C[c]$, y dentro del bloque las c están en el mismo orden que en L , así que la c de rango j ocupa la fila $C[c] + j$. Como la fila $LF(r)$ empieza justo con el carácter

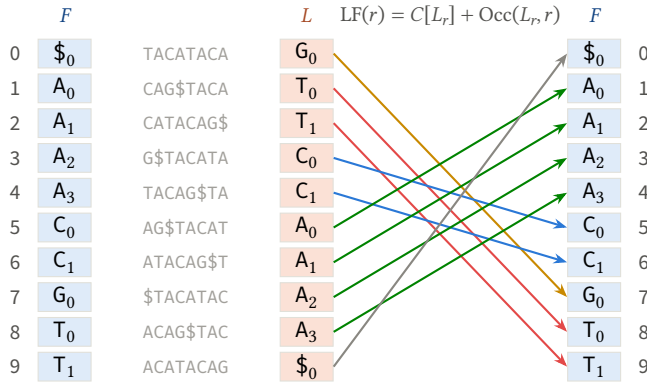


Figura 7.3. El LF-mapping para $T = \text{TACATACAG\$}$. Los subíndices numeran las apariciones de cada letra de arriba abajo. Cada flecha une la fila r de L con la fila $LF(r)$ de F (repetida a la derecha para mayor claridad): la j -ésima A de L va a la j -ésima A de F , y lo mismo para cada letra. Las flechas del mismo color nunca se cruzan; esa es la propiedad del teorema 7.1. En gris, el centro de cada fila, que el índice *no* almacena.

que precedía al sufijo de la fila r , aplicar LF equivale a *dar un paso hacia atrás* en el texto.

Ejemplo 7.1. (Reconstruir T desde su BWT)

Con $L = \text{GTTCCAAAA\$}$ y $F = \text{\$AAAACCGTT}$ tenemos $C[A] = 1$, $C[C] = 5$, $C[G] = 7$ y $C[T] = 8$. Empezamos en la fila 0, la que comienza con $\$$; su última letra, $L[0] = \text{G}$, es el último carácter de R . Aplicamos la ecuación (7.7) repetidamente:

$$0 \xrightarrow{\text{G}} 7 \xrightarrow{\text{A}} 3 \xrightarrow{\text{C}} 5 \xrightarrow{\text{A}} 1 \xrightarrow{\text{T}} 8 \xrightarrow{\text{A}} 4 \xrightarrow{\text{C}} 6 \xrightarrow{\text{A}} 2 \xrightarrow{\text{T}} 9,$$

donde sobre cada flecha escribimos $L[r]$ en la fila de partida. Por ejemplo, $LF(0) = C[G] + \text{Occ}(G, 0) = 7 + 0 = 7$ y $LF(7) = C[A] + \text{Occ}(A, 7) = 1 + 2 = 3$. Las letras recogidas, GACATACAT, leídas al revés dan TACATACAG: el texto original. En la fila 9 encontramos $L[9] = \$$ y paramos.

7.1.6 El índice FM y la búsqueda hacia atrás

El mismo mecanismo que reconstruye el texto sirve para *buscar* en él. La idea de Ferragina y Manzini (2005) es procesar el patrón $P = p_1 \cdots p_m$ de derecha a izquierda, manteniendo el intervalo de filas de M que empiezan con el sufijo de P ya procesado.

Teorema 7.2. (Búsqueda hacia atrás)

Sea $[sp, ep)$ el intervalo de filas de M que empiezan con la cadena W . Entonces las filas que empiezan con cW forman el intervalo

$$sp' = C[c] + \text{Occ}(c, sp), \quad ep' = C[c] + \text{Occ}(c, ep). \quad (7.8)$$

Empezando con W vacío, $[sp, ep) = [0, n)$, y aplicando la regla para $c = p_m, p_{m-1}, \dots, p_1$, el patrón aparece en T exactamente $ep - sp$ veces; si en algún paso $sp' \geq ep'$, no aparece.

DONDE

W	Sufijo del patrón ya procesado (al principio, la cadena vacía).
$[sp, ep)$	Intervalo semiabierto de filas de M (o del arreglo de sufijos) que empiezan con W .
c	Siguiente carácter del patrón, leído de derecha a izquierda.
$[sp', ep')$	Intervalo de las filas que empiezan con cW .

Por qué funciona. Las filas que empiezan con cW son exactamente las rotaciones $cW\dots$; al rotarlas un carácter a la izquierda se convierten en filas $W\dots c$, es decir, filas del intervalo $[sp, ep)$ cuya última letra es c . Por el teorema 7.1, esas c de L se mapean, en el mismo orden, a un bloque contiguo del bloque de c en F . Las c de L situadas antes de sp son $\text{Occ}(c, sp)$; por eso el nuevo bloque empieza en $C[c] + \text{Occ}(c, sp)$ y termina justo antes de $C[c] + \text{Occ}(c, ep)$. $\square$

Lo extraordinario es el coste: cada carácter del patrón cuesta dos consultas a Occ , sin importar el tamaño del genoma. Buscar una lectura de 150 bases en el genoma humano cuesta 300 consultas, lo mismo que buscarla en el genoma de un virus.

Ejemplo 7.2. (Buscar ACA en TACATACAG\$)

Usamos la tabla 7.1. Empezamos con $[0, 10)$ y leemos el patrón al revés:

Tabla 7.1. Tabla $\text{Occ}(c, r)$ para $L = \text{GTTCCAAAA}\$$: cada fila r cuenta las apariciones de cada base en $L[0..r-1]$. La tabla C es $C[A] = 1, C[C] = 5, C[G] = 7, C[T] = 8$.

r	$L[r]$	$\text{Occ}(A, r)$	$\text{Occ}(C, r)$	$\text{Occ}(G, r)$	$\text{Occ}(T, r)$
0	G	0	0	0	0
1	T	0	0	1	0
2	T	0	0	1	1
3	C	0	0	1	2
4	C	0	1	1	2
5	A	0	2	1	2
6	A	1	2	1	2
7	A	2	2	1	2
8	A	3	2	1	2
9	\$	4	2	1	2
10	–	4	2	1	2

1. $c = A$: $sp' = 1 + \text{Occ}(A, 0) = 1 + 0 = 1$; $ep' = 1 + \text{Occ}(A, 10) = 1 + 4 = 5$. Intervalo $[1, 5)$: las cuatro filas que empiezan con A.
2. $c = C$: $sp' = 5 + \text{Occ}(C, 1) = 5$; $ep' = 5 + \text{Occ}(C, 5) = 5 + 2 = 7$. Intervalo $[5, 7)$: filas que empiezan con CA.
3. $c = A$: $sp' = 1 + \text{Occ}(A, 5) = 1$; $ep' = 1 + \text{Occ}(A, 7) = 1 + 2 = 3$. Intervalo $[1, 3)$.

Hay $3 - 1 = 2$ apariciones, en las posiciones $SA[1] = 5$ y $SA[2] = 1$ del texto: TACATACAG. Si buscamos GAC, tras procesar AC tenemos $[1, 3)$ y el paso con G da $sp' = 7 + \text{Occ}(G, 1) = 8$ y $ep' = 7 + \text{Occ}(G, 3) = 8$: el intervalo se vacía y sabemos, sin mirar el texto, que GAC no aparece.

El siguiente programa construye el índice de forma ingenua (ordenando sufijos, lo que cuesta $O(n^2 \log n)$ y solo sirve para ejemplos) y busca por el método del teorema 7.2. Los programas reales construyen el arreglo de sufijos en tiempo lineal o casi lineal, con algoritmos especializados que no requieren tener en memoria todos los sufijos (Mäkinen et al., 2015).

	Paso 0: inicio	Paso 1: A	Paso 2: CA	Paso 3: ACA
0	\$TACATACAG	\$TACATACAG	\$TACATACAG	\$TACATACAG
1	ACAG\$TACAT	ACAG\$TACAT	ACAG\$TACAT	ACAG\$TACAT
2	ACATACAG\$T	ACATACAG\$T	ACATACAG\$T	ACATACAG\$T
3	AG\$TACATAC	AG\$TACATAC	AG\$TACATAC	AG\$TACATAC
4	ATACAG\$TAC	ATACAG\$TAC	ATACAG\$TAC	ATACAG\$TAC
5	CAG\$TACATA	CAG\$TACATA	CAG\$TACATA	CAG\$TACATA
6	CATACAG\$TA	CATACAG\$TA	CATACAG\$TA	CATACAG\$TA
7	G\$TACATACA	G\$TACATACA	G\$TACATACA	G\$TACATACA
8	TACAG\$TACA	TACAG\$TACA	TACAG\$TACA	TACAG\$TACA
9	TACATACAG\$	TACATACAG\$	TACATACAG\$	TACATACAG\$
	[0, 10) 10 filas	[1, 5) 4 filas	[5, 7) 2 filas	[1, 3) 2 filas

Figura 7.4. Búsqueda hacia atrás de $P = ACA$ en la matriz ordenada de $T = TACATACAG\$$, leída como una secuencia de fotogramas. En cada paso se antepone un carácter del patrón (de derecha a izquierda) y el intervalo amarillo se contrae a las filas que empiezan con el sufijo procesado, resaltado en rojo. El índice nunca almacena estas filas completas: solo necesita L , C y Occ para calcular los extremos del intervalo con la ecuación (7.8). El tamaño final del intervalo, dos filas, es el número de apariciones.

indice_fm.py

```
def indice_fm(T):
    """T debe terminar en '$'. Devuelve SA, L, C y Occ."""
    sa = sorted(range(len(T)), key=lambda i: T[i:])
    L = "".join(T[i - 1] for i in sa)          # T[-1] es '$'
    alf = sorted(set(T))
    C, tot = {}, 0
    for c in alf:
        C[c], tot = tot, tot + T.count(c)
    occ = {c: [0] for c in alf}
    for ch in L:                               # Occ(c, r) acumulado
        for c in alf:
            occ[c].append(occ[c][-1] + (ch == c))
    return sa, L, C, occ

def buscar(P, sa, C, occ):
    sp, ep = 0, len(sa)
    for c in reversed(P):                       # de derecha a izquierda
        if c not in C:
            return []
        sp, ep = C[c] + occ[c][sp], C[c] + occ[c][ep]
```

```

    if sp >= ep:
        return []
    return sorted(sa[sp:ep])

sa, L, C, occ = indice_fm("TACATACAG$")
print(L, buscar("ACA", sa, C, occ))  # GTTCCAAAA$ [1, 5]

```

7.1.7 Muestreo: cómo cabe el genoma humano en un portátil

El programa anterior guarda Occ completa: cuatro enteros por posición. Para el genoma humano, con enteros de 32 bits, eso son $4 \times 32 \times 3,1 \times 10^9$ bits, unos 50 GB, peor que el arreglo de sufijos. El índice FM resuelve el problema con dos muestreos (Ferragina & Manzini, 2005).

Puntos de control de Occ. Se guardan los valores de $\text{Occ}(c, r)$ solo cuando r es múltiplo de un intervalo d (los *checkpoints*). Para cualquier otra fila, se toma el punto de control anterior y se cuentan las apariciones de c en el trozo de L que falta, a lo sumo $d - 1$ caracteres. Con L codificada a 2 bits por base, ese recuento se hace con operaciones de bits sobre unas pocas palabras de máquina.

Muestreo del arreglo de sufijos. La búsqueda hacia atrás devuelve un intervalo de *filas*, pero necesitamos *posiciones* del genoma, es decir, valores de SA. Se guarda $\text{SA}[r]$ solo para las filas cuyo valor es múltiplo de s . Para una fila no muestreada, aplicamos LF (un paso atrás en el texto) hasta caer en una fila muestreada; si hicieron falta j pasos, $\text{SA}[r] = \text{SA}[\text{LF}^j(r)] + j$, con $j < s$.

La memoria total, en bits, es

$$\mathcal{M}(d, s) = \frac{2n}{\tilde{L}} + \underbrace{\frac{4bn}{d}}_{\text{Occ}} + \underbrace{\frac{bn}{s}}_{\text{SA}}, \quad (7.9)$$

mientras que el tiempo de localizar cada aparición crece como $O(s)$ pasos de LF, cada uno de coste $O(d)$.

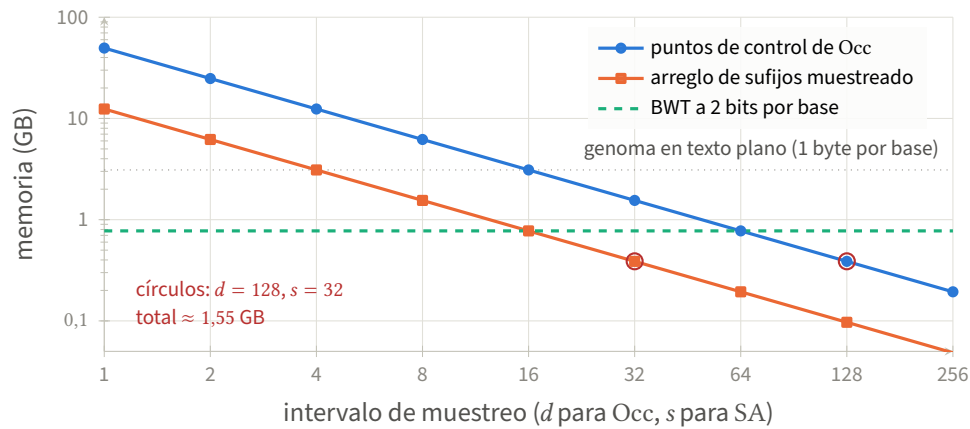


Figura 7.5. Memoria de cada componente del índice FM del genoma humano ($n = 3,1 \times 10^9$, enteros de 32 bits) en función del intervalo de muestreo (ecuación (7.9)). Sin muestreo, Occ ocuparía unos 50 GB y el arreglo de sufijos 12 GB. Con un punto de control cada 128 filas y una muestra del arreglo cada 32 posiciones (círculos), el índice completo de una hebra ocupa unos 1,55 GB, la mitad que el genoma escrito como texto. El precio es tiempo: cada posición recuperada exige, en promedio, varios pasos de *LF-mapping*.

DONDE	
$\mathcal{M}(d, s)$	Memoria del índice, en bits.
n	Longitud del texto indexado.
b	Bits por entero almacenado ($b = \lceil \log_2 n \rceil = 32$ para el genoma humano).
d	Distancia entre puntos de control de Occ.
s	Intervalo de muestreo del arreglo de sufijos.

Con $d = 128$ y $s = 32$, la ecuación (7.9) da 0,775 GB para L , 0,388 GB para los puntos de control y otros 0,388 GB para el arreglo muestreado: unos 1,55 GB (figura 7.5). Estos son, de hecho, los intervalos que describen Li y Durbin (2009) para BWA, que además indexa el genoma invertido para poder acotar la búsqueda aproximada; Bowtie, con técnicas similares, alineaba más de 25 millones de lecturas por hora de CPU usando unos 1,3 GB de memoria (Langmead et al., 2009).

Buscar con errores: *backtracking* acotado

La búsqueda hacia atrás encuentra coincidencias *exactas*, pero una lectura real contiene errores de secuenciación y variantes. Bowtie y la primera versión de BWA exploran variantes del patrón: en cada paso, además de la base leída, prueban las otras tres (una sustitución) o saltan un carácter (un *indel*), y retroceden cuando el intervalo se vacía. El espacio de búsqueda crece exponencialmente con el número de diferencias permitidas, así que ambas herramientas lo podan. Bowtie prioriza sustituir primero las bases de baja calidad Phred (Langmead et al., 2009). BWA precalcula un arreglo $D(i)$, cota inferior del número de diferencias que necesitará el prefijo $P[0..i]$, y abandona una rama en cuanto las diferencias acumuladas más $D(i)$ superan el máximo permitido (Li & Durbin, 2009). Un detalle elegante: como los sufijos repetidos del genoma comparten intervalo, una repetición con mil copias se explora *una sola vez*.

Detalles que el ejemplo oculta

- *Las dos hebras*. Una lectura puede proceder de cualquier hebra: el mapeador busca q y su reverso complementario $\bar{q}$, o indexa ambas hebras de la referencia.
- *Las N*. Las bases ambiguas de la referencia (huecos del ensamblaje) se sustituyen por bases aleatorias o se excluyen al construir el índice; en la lectura se tratan como diferencias.
- *Varios cromosomas*. Se concatenan en un único texto; una tabla auxiliar traduce la posición global a (cromosoma, posición) y descarta las coincidencias que cruzan una frontera.

La BWT reordena el genoma de modo que buscar un patrón cuesta dos consultas por carácter, independientemente del tamaño del genoma, y el índice completo cabe en menos memoria que el propio texto.

7.2 BWA, minimap2 y samtools

Huellas digitales de un texto copiado

Un profesor sospecha que un ensayo fue copiado de alguno de los millones de documentos de internet. No puede compararlo palabra por palabra con todos. En cambio, toma de cada documento unas pocas “huellas”: por ejemplo, en cada tramo de diez frases, la que tenga el código más pequeño según una regla fija. Dos textos que compartan un párrafo largo compartirán, por fuerza, algunas huellas, porque la regla elige la misma frase en ambos. Con las huellas coincidentes en la mano, el profesor comprueba si aparecen en el mismo orden y a distancias parecidas; si es así, ha encontrado la fuente, y solo entonces lee con atención el pasaje sospechoso. Semilla, encadenamiento y extensión: así trabajan los mapeadores modernos.

7.2.1 El paradigma semilla-encadenamiento-extensión

La búsqueda exacta con retroceso funciona bien para lecturas de 36 bases con dos o tres diferencias, las de los primeros secuenciadores Illumina. Con lecturas de 100 a 250 bases, y más aún con lecturas largas de nanoporos, la búsqueda con errores se vuelve inviable y, además, innecesaria: una lectura larga contiene, casi con seguridad, varios tramos exactos suficientemente largos para ser únicos. Los mapeadores actuales combinan tres etapas:

1. **Semillas:** coincidencias exactas entre la lectura y la referencia, halladas con un índice (FM o *hash*).
2. **Encadenamiento:** agrupar las semillas que son *colineales* (mismo orden y distancias compatibles en lectura y referencia), porque proceden probablemente de un mismo origen.
3. **Extensión:** alinear con programación dinámica, restringida a una banda, las bases entre semillas y los extremos de la cadena.

La programación dinámica, cara, solo se ejecuta en un puñado de regiones candidatas por lectura.

BWA-MEM. La familia BWA pasó por tres algoritmos: BWA-*backtrack*, para lecturas cortas (Li & Durbin, 2009); BWA-SW, que alineaba lecturas largas recorriendo el

índice con programación dinámica (Li & Durbin, 2010); y BWA-MEM (Li, 2013), hoy el estándar para lecturas Illumina. BWA-MEM siembra con *coincidencias exactas supermaximales* (SMEM): para cada posición de la lectura, la coincidencia exacta más larga que la cubre y que no está contenida en otra. El índice FM es ideal para esto, porque extender una coincidencia un carácter cuesta una actualización del intervalo. Si una SMEM es muy larga (más de 28 bases por defecto), BWA-MEM vuelve a sembrar con coincidencias más cortas que aparecen más veces, para no perder el origen verdadero cuando la SMEM cae en una repetición. Después encadena semillas cercanas y colineales, descarta cadenas contenidas en otras mucho mejores, y extiende con programación dinámica de huecos afines en banda. Durante la extensión aplica una variante del criterio *X-drop* de BLAST que vimos en el ??, y decide automáticamente entre alineamiento local y de extremo a extremo.

Bowtie 2. Langmead y Salzberg (2012) siguieron un camino paralelo: el índice FM encuentra semillas sin huecos y la extensión se hace con programación dinámica vectorizada con instrucciones SIMD del procesador. Bowtie 2 sigue siendo muy usado en ChIP-seq y ATAC-seq.

7.2.2 Minimizadores

Para lecturas largas y ruidosas, Li (2018) optó en minimap2 por un índice de *hash*, pero sin guardar todos los k -meros del genoma: solo sus *minimizadores*, una idea de Roberts et al. (2004).

Definición 7.4. $((w, k)$ -minimizador)

Sea h una función que ordena los k -meros (por ejemplo, un *hash* pseudo-aleatorio). Para cada ventana de w k -meros consecutivos de una secuencia, el minimizador de la ventana es el k -mero de valor h mínimo:

$$\mu_j = \arg \min_{j \leq i < j+w} h(s[i..i+k-1]). \quad (7.10)$$

El conjunto de minimizadores de la secuencia es $\{\mu_j\}$ para todas las ventanas j .

DONDE

k	Longitud de cada k -mero.
w	Número de k -meros consecutivos por ventana; la ventana abarca $w + k - 1$ bases.
h	Orden total sobre los k -meros; el orden alfabético es el más simple, un <i>hash</i> aleatorio el más eficaz.
μ_j	Posición del k -mero elegido en la ventana que empieza en j .

La definición garantiza dos propiedades. La primera es la de *cobertura*: toda ventana de $w + k - 1$ bases aporta al menos un minimizador, así que dos secuencias que comparten un tramo exacto de esa longitud comparten al menos un minimizador; es la misma garantía que buscaba el algoritmo de *winnowing* para detectar plagio en documentos (Schleimer et al., 2003). La segunda es la de *escasez*: ventanas vecinas comparten casi todos sus k -meros y suelen elegir el mismo, de modo que el número de minimizadores distintos es mucho menor que el de k -meros.

Teorema 7.3. (*Densidad de los minimizadores*)

Si los k -meros de una secuencia aleatoria reciben valores h independientes y sin empates, la fracción esperada de posiciones elegidas como minimizador es

$$\delta \approx \frac{2}{w + 1}. \quad (7.11)$$

DONDE

δ	Densidad: número de minimizadores dividido por el número de k -meros.
----------	---

Esbozo. Compare dos ventanas consecutivas, j y $j + 1$. Juntas abarcan $w + 1$ k -meros, y por simetría cada uno tiene la misma probabilidad, $1/(w + 1)$, de ser el menor de todos. Si el menor es el primero (que solo pertenece a la ventana j), la ventana $j + 1$ tiene que elegir otro k -mero: aparece un minimizador nuevo. Si el menor es el último (el que acaba de entrar), la ventana $j + 1$ lo elige y también es nuevo. Si está en cualquier posición intermedia, ambas ventanas eligen el mismo y no se añade nada. La probabilidad de estrenar minimizador al avanzar una posición es, por tanto, $2/(w + 1)$ (Schleimer et al., 2003). $\square$

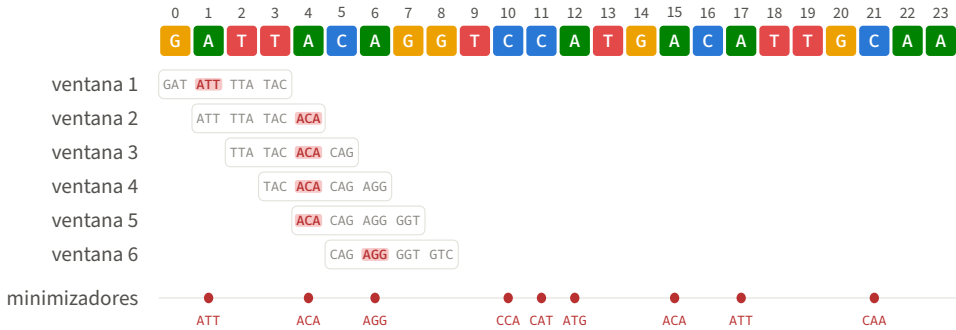


Figura 7.6. Minimizadores ($w = 4$, $k = 3$) de una secuencia de 24 bases, con orden alfabético para que puedan comprobarse a mano. Cada fila muestra una ventana de cuatro 3-meros consecutivos, cada uno escrito bajo su base inicial, con el menor resaltado en rojo. Las ventanas 2 a 5 eligen el mismo ACA: por eso se guardan muchos menos minimizadores que k -meros. La última fila marca los nueve minimizadores de toda la secuencia (de 22 k -meros; densidad 0,41, frente a $2/(w + 1) = 0,40$). Una lectura que contenga cualquier tramo de $w + k - 1 = 6$ bases de esta secuencia compartirá con ella al menos uno de estos minimizadores.

Con $w = 10$ se guarda solo un k -mero de cada 5,5: el índice se reduce a menos de una quinta parte. En una secuencia aleatoria de 200 000 bases con $k = 15$ y $w = 10$, nuestra simulación da una densidad de 0,188 con un *hash* aleatorio (la teoría predice 0,182) y de 0,199 con orden alfabético, que tiende a elegir k -meros ricos en A consecutivos y es menos eficiente.

minimizadores.py

```
import zlib

def minimizadores(s, k=15, w=10):
    """Posiciones de los (w,k)-minimizadores de s."""
    h = [zlib.crc32(s[i:i + k].encode())          # orden pseudoaleatorio
          for i in range(len(s) - k + 1)]
    elegidos = set()
    for j in range(len(h) - w + 1):
        v = h[j:j + w]
        elegidos.add(j + v.index(min(v)))
    return sorted(elegidos)
```

7.2.3 Encadenamiento

Cada minimizador de la lectura que también aparece en el índice produce un *ancla*: un par de posiciones (x, y) en referencia y lectura donde termina una coincidencia exacta de longitud w_i (aquí, $w_i = k$). Las anclas verdaderas se alinean a lo largo de una diagonal; las espurias, procedentes de repeticiones, quedan dispersas. Li (2018) define la puntuación $f(i)$ de la mejor cadena que termina en el ancla i mediante programación dinámica:

$$f(i) = \max \left\{ \max_{j < i} \{f(j) + \alpha(j, i) - \beta(j, i)\}, w_i \right\}, \quad (7.12)$$

con

$$\alpha(j, i) = \min \{ \min \{y_i - y_j, x_i - x_j\}, w_i \}, \quad \beta(j, i) = \gamma((y_i - y_j) - (x_i - x_j)), \quad (7.13)$$

$$\gamma(\ell) = \begin{cases} 0,01 \bar{w} |\ell| + 0,5 \log_2 |\ell| & \ell \neq 0, \\ 0 & \ell = 0. \end{cases} \quad (7.14)$$

DONDE

(x_i, y_i, w_i)	Ancla i : termina en la posición x_i de la referencia y y_i de la lectura, y cubre w_i bases idénticas.
$f(i)$	Puntuación máxima de una cadena de anclas que termina en i .
$\alpha(j, i)$	Bases coincidentes que el ancla i añade tras la j (no cuenta dos veces el solapamiento).
$\beta(j, i)$	Penalización por el desfase entre las distancias en lectura y referencia: un <i>indel</i> de longitud $ \ell $.
$\bar{w}$	Longitud media de las semillas.

β vale infinito si el ancla j no precede a la i en la lectura o si están demasiado lejos. La ecuación (7.12) es una recurrencia de tipo Smith-Waterman (compárela con la del ??) sobre *anclas* en lugar de bases: el término w_i permite empezar una cadena nueva, igual que el cero permitía empezar un alineamiento local. En la práctica se evalúan solo los 50 predecesores más cercanos, lo que reduce el coste de cuadrático a lineal en el número de anclas sin perder casi nunca la cadena óptima.

Tras encadenar, minimap2 rellena con programación dinámica los huecos entre anclas consecutivas y los extremos, con una penalización de huecos “afín en dos tramos” que permite *indels* largos. El mismo programa, con distintos parámetros preestablecidos (-x sr para lecturas cortas, map-ont y map-hifi para lecturas largas, splice para ARN, asm5 para ensamblajes), cubre casi todos los casos de uso.

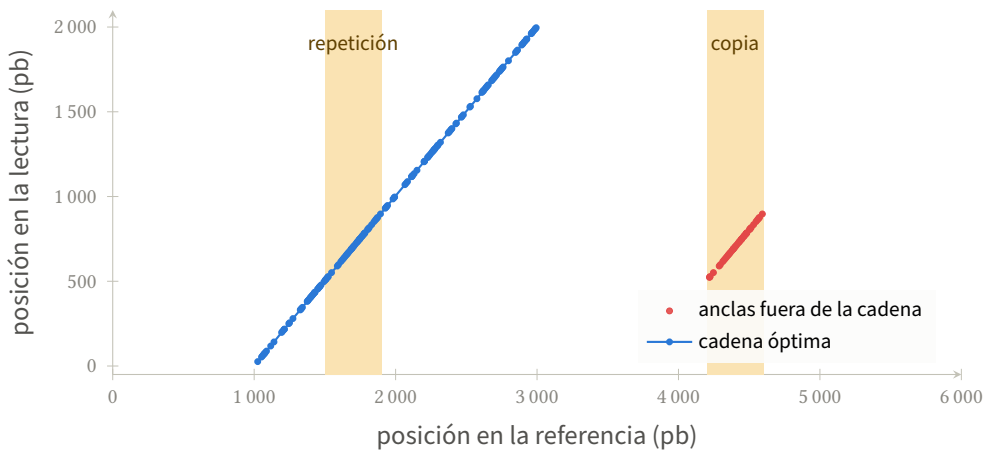


Figura 7.7. Encadenamiento de minimizadores ($k = 15$, $w = 10$) de una lectura simulada de 2000 pb, con un 4 % de errores, contra una referencia de 6000 pb que contiene una repetición de 400 pb (bandas amarillas). La ecuación (7.12) elige 187 de las 236 anclas (azul), alineadas sobre una diagonal casi perfecta: la lectura procede de las posiciones 1000 a 3000. Las anclas rojas del extremo derecho son minimizadores de la repetición que también aparecen en su copia; no pueden encadenarse con el resto sin pagar un *indel* de casi 2700 bases, y quedan descartadas. Si la lectura cayera *entera* dentro de la repetición, las dos cadenas empatarían: es el origen de la ambigüedad que mide la calidad de mapeo.

Índices y mapeo

```
# BWA-MEM: índice FM (una sola vez) y mapeo de lecturas pareadas
bwa index ref.fa
bwa mem -t 8 -R '@RG\tID:m1\tSM:m1' ref.fa r1.fq.gz r2.fq.gz \
> m1.sam
# minimap2: índice de minimizadores al vuelo (o guardado con -d)
minimap2 -ax sr ref.fa r1.fq.gz r2.fq.gz > m1_sr.sam
minimap2 -ax map-ont ref.fa nanoporo.fq.gz > m1_ont.sam
```

7.2.4 Calidad de mapeo

Supongamos que una lectura tiene dos ubicaciones candidatas. ¿Cuán seguros podemos estar de la mejor? Li et al. (2008) introdujeron en MAQ la *calidad de mapeo*, que expresa en escala Phred (la misma del ??) la probabilidad de que la posición asignada sea errónea.

Definición 7.5. (Calidad de mapeo)

Sean $u_1, \dots, u_K$ las ubicaciones candidatas de una lectura q y $p(q | u)$ la probabilidad de observar q si procede de u . Suponiendo un origen *a priori* uniforme, la probabilidad *a posteriori* de que la mejor ubicación $\hat{u}$ sea la verdadera es

$$\mathbb{P}(\hat{u} | q) = \frac{p(q | \hat{u})}{\sum_{k=1}^K p(q | u_k)}, \quad (7.15)$$

y la calidad de mapeo es

$$\text{MAPQ} = -10 \log_{10} (1 - \mathbb{P}(\hat{u} | q)). \quad (7.16)$$

DONDE

u_k	Posición candidata k (con su hebra).
$p(q u)$	Verosimilitud de la lectura dada la posición; si solo hay sustituciones, aproximadamente el producto de las probabilidades de error $\varepsilon_i = 10^{-Q_i/10}$ de las bases discordantes.
MAPQ	Calidad de mapeo en escala Phred: 20 significa 1 % de probabilidad de posición errónea, 30 significa 0,1 %.

Ejemplo 7.3. (Dos candidatas y un empate)

Una lectura coincide con la posición u_1 salvo por una base de calidad $Q = 35$, y con u_2 salvo por dos bases de calidades 25 y 30. Aproximando $p(q | u)$ por el producto de las probabilidades de error de las discordancias,

$$p(q | u_1) = 10^{-3,5} = 3,16 \times 10^{-4}, \quad p(q | u_2) = 10^{-2,5} \cdot 10^{-3} = 3,16 \times 10^{-6}.$$

La probabilidad de error es $3,16 \times 10^{-6} / (3,16 \times 10^{-4} + 3,16 \times 10^{-6}) = 0,0099$, y $\text{MAPQ} = -10 \log_{10} 0,0099 = 20,0$. Una sola base discordante de diferencia, pero de buena calidad, basta para preferir u_1 con un 99 % de confianza. Si en cambio u_1 y u_2 fueran idénticas, la ecuación (7.16) daría $-10 \log_{10} 0,5 = 3,01$; con tres copias idénticas, 1,76.

En la práctica, los programas no enumeran todas las candidatas: estiman el denominador a partir de las dos mejores y de cuántas subóptimas encontraron. BWA supone que la ubicación verdadera siempre está entre las halladas y comprueba por simulación que sus valores son razonables: de 1 569 108 lecturas simuladas con $\text{MAPQ} = 60$, solo 11 estaban mal colocadas (Li & Durbin, 2009). minimap2 usa una fórmula empírica basada en las puntuaciones de encadenamiento de la cadena primaria (f_1) y de la mejor secundaria (f_2):

$$\text{MAPQ} = 40 \left(1 - \frac{f_2}{f_1} \right) \min \left\{ 1, \frac{a}{10} \right\} \ln f_1, \quad (7.17)$$

donde a es el número de anclas de la cadena primaria (Li, 2018); el valor final se trunca en 60. Con $f_1 = 200$ y 40 anclas, una secundaria con $f_2 = 150$ da un MAPQ de 53, y una casi empatada con $f_2 = 195$ lo hunde hasta 5.

⚠ Lo que el MAPQ no es

- *No es una calidad del alineamiento.* Una lectura puede alinearse perfectamente, sin una sola discordancia, y tener $\text{MAPQ} = 0$ porque encaja igual de bien en dos copias de una repetición.
- *No es comparable entre programas.* BWA, Bowtie 2 y minimap2 calculan y truncan el MAPQ de maneras distintas. Un filtro “ $\text{MAPQ} \geq 20$ ” significa cosas diferentes en cada caso.

- *Depende de la referencia.* Si falta en la referencia la copia verdadera de una repetición, las lecturas se mapean con confianza al lugar equivocado.

7.2.5 Repeticiones y multimap

Casi la mitad del genoma humano está formada por secuencias repetidas: elementos transponibles, satélites, duplicaciones segmentarias (Treangen & Salzberg, 2012). Una lectura de 150 bases que cae en el interior de una copia reciente de un elemento Alu o de una duplicación segmentaria no tiene un origen identificable: se *multimapea*. Los formatos distinguen tres tipos de registros: el alineamiento *primario*, uno por lectura, elegido al azar entre los empatados; los *secundarios*, que señalan otras ubicaciones plausibles; y los *suplementarios*, que no son alternativas sino *piezas* de una lectura quimérica, cuyo extremo izquierdo se alinea en un lugar y el derecho en otro, como ocurre en una translocación.

Cuánto de este problema es inevitable depende también de la referencia. GRCh38 dejó, pese a su calidad (Schneider et al., 2017), huecos en centrómeros y brazos cortos acrocéntricos. El genoma T2T-CHM13 completó el 8 % restante: 3055 millones de bases sin huecos, con casi 200 millones de bases nuevas, en buena parte satélites y duplicaciones segmentarias recientes (Nurk et al., 2022). Las lecturas que antes no tenían dónde caer, o que caían en una copia paráloga equivocada, ahora tienen un sitio.

7.2.6 Lecturas pareadas y tamaño de inserto

Cuando se secuencian ambos extremos de un fragmento, las dos lecturas no son independientes: deben mapearse en hebras opuestas, orientadas una hacia la otra, a una distancia compatible con el tamaño de los fragmentos de la biblioteca. Esa información resuelve muchas ambigüedades: si una lectura cae en una repetición pero su pareja cae en secuencia única, la pareja “ancla” a la primera. BWA-MEM estima la distribución del tamaño de inserto a partir de los pares mapeados sin ambigüedad y puntúa cada combinación (i, j) de ubicaciones de las dos lecturas con (Li, 2013)

$$S_{ij} = S_i + S_j - \min \{ -a \log_4 P(d_{ij}), U \}, \quad (7.18)$$

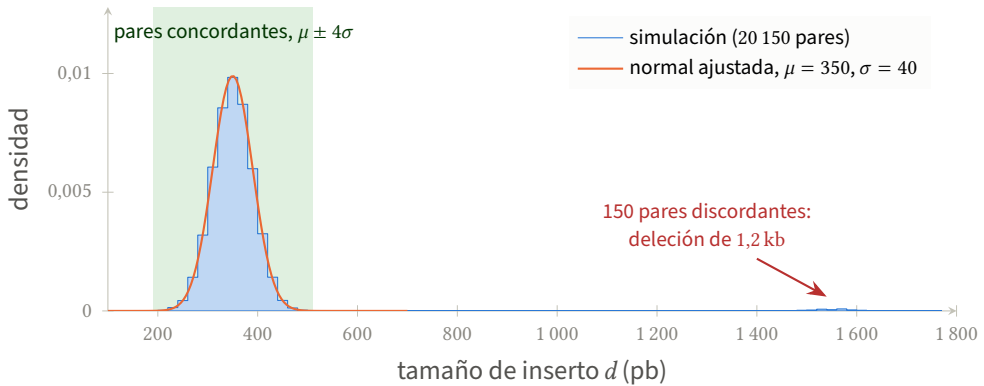


Figura 7.8. Tamaño de inserto de 20 000 pares simulados con $\mu = 350$ y $\sigma = 40$ pb, más 150 pares que atraviesan una deleción de 1200 pb en la muestra. Como la referencia conserva el segmento que la muestra perdió, esos pares parecen 1200 pb más largos. Bajo el modelo normal, la probabilidad de un inserto mayor que $\mu + 4\sigma = 510$ pb es 3×10^{-5} : esperaríamos menos de un par así por azar, y observamos 150 agrupados en la misma región. Los detectores de variantes estructurales combinan esta señal con la cobertura (sección 7.3) y con las lecturas partidas.

DONDE

S_i, S_j	Puntuaciones de Smith-Waterman de cada lectura en su ubicación.
d_{ij}	Distancia entre las dos ubicaciones (infinita si la orientación es incorrecta).
$P(d)$	Probabilidad de observar un inserto mayor que d según una distribución normal ajustada.
a	Puntuación de una coincidencia; convierte probabilidades en unidades de puntuación.
U	Penalización máxima: si el par es muy improbable, es preferible tratar las lecturas como no emparejadas.

El logaritmo en base 4 aparece al interpretar la puntuación de Smith-Waterman como una razón de verosimilitudes, como vimos en el ???. Los pares que el modelo no puede explicar, por estar demasiado separados, demasiado juntos o mal orientados, son *discordantes*, y son precisamente la señal que buscan los detectores de variantes estructurales (figura 7.8).

Tabla 7.2. Los once campos obligatorios de un registro SAM, ilustrados con el primer extremo de un par ficticio de 20 bases. Las posiciones son base 1.

#	Campo	Ejemplo	Significado
1	QNAME	r017	Nombre de la lectura (igual en ambos extremos del par).
2	FLAG	99	Suma de bits (tabla 7.3): 1 + 2 + 32 + 64.
3	RNAME	chr7	Secuencia de referencia.
4	POS	1021	Posición de la primera base <i>alineada</i> (no recortada).
5	MAPQ	60	Calidad de mapeo (ecuación (7.16)); 255 si no está disponible.
6	CIGAR	3S12M1D5M	Operaciones del alineamiento: 3 recortadas, 12 alineadas, 1 delección, 5 alineadas.
7	RNEXT	=	Referencia de la pareja (=: la misma).
8	PNEXT	1180	Posición de la pareja.
9	TLEN	179	Longitud observada del fragmento, de 1021 a 1199.
10	SEQ	GTA...	Bases de la lectura, en la hebra de la referencia.
11	QUAL	FFF...	Calidades Phred+33 (??).

7.2.7 El formato SAM/BAM

Todos los mapeadores escriben sus resultados en el formato SAM (*Sequence Alignment/Map*), creado para el Proyecto 1000 Genomas (Li et al., 2009) y mantenido hoy como especificación formal (SAM/BAM Format Specification Working Group, 2024). Un archivo SAM es texto tabulado: una cabecera con líneas que empiezan por @ (secuencias de referencia @SQ, grupos de lectura @RG, programas @PG) y una línea por alineamiento con once campos obligatorios (tabla 7.2), seguidos de etiquetas opcionales TAG:TIPO:VALOR, como NM:i:2 (distancia de edición) o AS:i: (puntuación del alineamiento).

La cadena CIGAR. Describe el alineamiento como una sucesión de longitudes y operaciones: M (alineada, coincida o no; = y X distinguen ambos casos), I (inserción respecto a la referencia), D (delección), N (salto en la referencia, típico de un intrón en RNA-seq), S (recorte suave: las bases están en SEQ pero no alineadas) y H (recorte duro: las bases ni siquiera se guardan). En el ejemplo, las operaciones que consumen la lectura (S, M, I) suman 3 + 12 + 5 = 20, su longitud, y las que consumen referencia (M, D, N) suman 12 + 1 + 5 = 18: el alineamiento cubre las posiciones 1021 a 1038. Si la pareja se alinea como 20M a partir de 1180, termina en 1199 y el fragmento mide 1199 – 1021 + 1 = 179 pb.

Tabla 7.3. Bits del campo FLAG de SAM (SAM/BAM Format Specification Working Group, 2024). Un filtro frecuente es -F 0x904 (excluir no mapeadas, secundarias y suplementarias), que deja un registro primario por lectura mapeada.

Bit	Dec.	Significado	Bit	Dec.	Significado
0x1	1	lectura pareada	0x40	64	primera del par
0x2	2	par concordante (<i>proper pair</i>)	0x80	128	segunda del par
0x4	4	lectura no mapeada	0x100	256	alineamiento secundario
0x8	8	pareja no mapeada	0x200	512	no pasa filtros de calidad
0x10	16	lectura en hebra reversa	0x400	1024	duplicado de PCR u óptico
0x20	32	pareja en hebra reversa	0x800	2048	alineamiento suplementario

BAM es la versión binaria y comprimida de SAM, en bloques BGZF independientes; eso permite *indexarla* (archivo .bai o .csi) y saltar directamente a una región sin leer el resto del archivo, siempre que esté ordenada por coordenada. CRAM va más allá y guarda solo las diferencias respecto a la referencia, con archivos aún menores.

7.2.8 samtools: del SAM crudo al BAM analizable

El SAM que sale del mapeador está en el orden de las lecturas del FASTQ. Para analizarlo hacen falta varios pasos, que `samtools` resuelve en un flujo de tuberías (Danecek et al., 2021; Li et al., 2009). El orden importa (figura 7.9):

1. `samtools fixmate -m` rellena, con los dos extremos de cada par a la vista, la información sobre la pareja y añade las etiquetas que necesitará el marcado de duplicados. Requiere los pares contiguos, como los entrega BWA-MEM.
2. `samtools sort` ordena por coordenada.
3. `samtools markdup` marca con el bit 0x400 los *duplicados*: pares cuyos extremos 5' coinciden en posición y orientación, que probablemente son copias por PCR de un mismo fragmento original o duplicados ópticos del secuenciador (recuerde la complejidad de la biblioteca del ??). `samblaster` lo hace en flujo, sin ordenar, y además extrae las lecturas discordantes y partidas (Faust & Hall, 2014).
4. `samtools index` crea el índice para el acceso aleatorio.

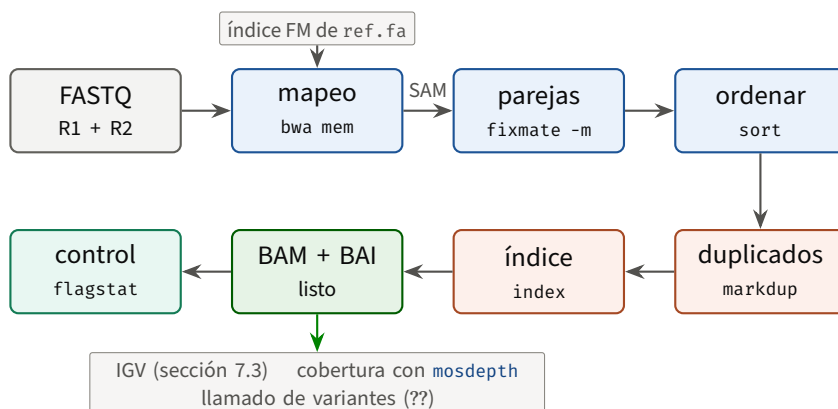


Figura 7.9. Flujo de trabajo típico de mapeo de lecturas Illumina pareadas. En azul, los pasos que se encadenan en una sola tubería sin archivos intermedios; en naranja, los que operan sobre el BAM ordenado. `fixmate` debe ir *antes* de ordenar, cuando los dos extremos de cada par aún están contiguos, y `markdup`, *después*. El BAM indexado es el punto de partida de la visualización, la cobertura y el llamado de variantes.

De SAM a BAM analizable

```

bwa mem -t 8 -R '@RG\tID:m1\tSM:m1' ref.fa r1.fq.gz r2.fq.gz \
| samtools fixmate -m - - \
| samtools sort -@ 4 -o m1.ordenado.bam -
samtools markdup -@ 4 m1.ordenado.bam m1.bam
samtools index m1.bam
samtools flagstat m1.bam # resumen de FLAG
samtools view -c -F 0x904 -q 20 m1.bam # primarias con MAPQ>=20
samtools view m1.bam chr7:55019017-55211628 | head # una región
  
```

⚠ Errores frecuentes en el flujo

- Olvidar el grupo de lectura (-R). Muchos programas posteriores lo exigen para distinguir muestras y carriles.
- Marcar duplicados en datos de amplicones o de RNA-seq sin pensarlo: allí los “duplicados” son la norma biológica o técnica, no un artefacto.

- Mezclar referencias: el BAM solo es válido junto con la misma versión exacta del FASTA (chr7 frente a 7, GRCh37 frente a GRCh38). Consulte las líneas @SQ de la cabecera con `samtools view -H`.

7.3 Visualización de alineamientos y cobertura



Colab · 7.3 · Visualización de alineamientos y cobertura
curso



repositorio del

Una autopista vista desde un helicóptero

Desde un helicóptero se ve el tráfico de una autopista de dos carriles como una densidad de coches más o menos uniforme. Si un tramo tiene un carril cerrado por obras, la densidad cae a la mitad; si el tramo entero está cortado, no pasa ningún coche; si se abre un tercer carril, la densidad sube un cincuenta por ciento. Y hay un tramo en que la autopista corre paralela a otra idéntica: desde el aire es imposible saber por cuál va cada coche, y el contador automático, prudente, no cuenta ninguno. Un perfil de cobertura es esa vista aérea del genoma: las lecturas son los coches, el número de copias son los carriles, y las repeticiones son las autopistas gemelas.

7.3.1 Mirar antes de calcular: IGV

Así como el *dot plot* del ?? precedía al alineamiento, mirar los alineamientos precede a cualquier análisis serio. El *Integrative Genomics Viewer* (IGV) permite navegar un BAM indexado a cualquier escala, desde un cromosoma entero hasta una base (Robinson et al., 2011). Su diseño carga solo la región visible, aprovechando el índice del BAM, y muestra la información en *pistas* apiladas: la anotación de genes, la cobertura, las lecturas individuales (Thorvaldsdóttir et al., 2013). Las convenciones visuales codifican casi todos los campos del SAM:

- las bases que *coinciden* con la referencia no se dibujan; solo se colorean las discordantes, de modo que el ojo va directo a las diferencias;
- una línea negra horizontal indica una delección, y una marca violeta, una inserción;

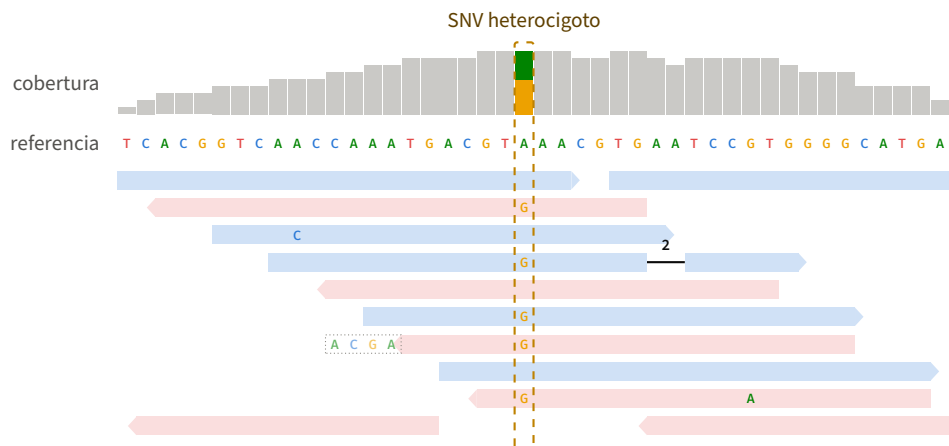


Figura 7.10. Apilamiento (*pileup*) de lecturas sobre 44 bases de referencia, dibujado con las convenciones de IGV. Las lecturas azules se alinean en la hebra directa y las rojas en la reversa (la punta indica el sentido). En la columna recuadrada, 5 de las 9 lecturas llevan una G donde la referencia tiene A: la pista de cobertura se colorea con esa proporción (0,56), la firma de un SNV heterocigoto. Las letras aisladas en otras columnas (una C y una A) son errores de secuenciación: aparecen en una sola lectura. La línea negra marca una delección de 2 pb en una lectura, y las letras tenues dentro del recuadro punteado son bases con recorte suave (4S), presentes en la lectura pero no alineadas.

- el color de fondo de cada lectura puede indicar su hebra, y las lecturas con $\text{MAPQ} = 0$ se dibujan transparentes;
- en la pista de cobertura, las posiciones donde una fracción notable de las lecturas difiere de la referencia se colorean con la proporción de cada base.

La figura 7.10 resume la lectura de un *pileup*. Distinguir una variante de un error es una cuestión de conteo. Si la tasa de error por base es $\varepsilon = 10^{-3}$ (calidad 30), la probabilidad de que 5 o más de 9 lecturas muestren la misma base errónea por azar es del orden de $\binom{9}{5}\varepsilon^5 \approx 1,3 \times 10^{-13}$; la de que una sola lectura muestre un error, $9\varepsilon \approx 0,009$. El siguiente capítulo convertirá este razonamiento en un modelo bayesiano de genotipos.

- *Sesgo de hebra*: una “variante” presente solo en lecturas de una hebra suele ser un artefacto químico o de alineamiento.
- *Variantes al final de las lecturas*: cerca de un *indel*, el alineador puede preferir varias sustituciones en el extremo de la lectura a abrir un hueco, lo que produce falsos SNV agrupados.
- *Pilas de lecturas con MAPQ bajo y muchas discordancias*: huella de una repetición o de una región que falta en la referencia; las lecturas de otro lugar “se refugian” allí.

7.3.2 La cobertura como señal

La definición de cobertura del ?? era un promedio; ahora la necesitamos posición a posición.

Definición 7.6. (Profundidad por posición)

Sea $\mathcal{A}$ el conjunto de alineamientos que superan los filtros (por ejemplo, primarios, no duplicados y con $\text{MAPQ} \geq 20$), y sea $B(a)$ el conjunto de bloques de referencia $[s, e)$ que el alineamiento a cubre, es decir, sus operaciones M , $=$ o X de la cadena CIGAR. La profundidad en la posición x es

$$c(x) = \sum_{a \in \mathcal{A}} \sum_{[s,e) \in B(a)} \mathbb{1}[s \leq x < e]. \quad (7.19)$$

DONDE

$c(x)$	Número de lecturas filtradas que cubren la posición x con una base alineada.
$\mathcal{A}$	Alineamientos que se cuentan; los filtros cambian mucho el resultado en repeticiones.
$[s, e)$	Bloque semiabierto de posiciones de referencia cubierto sin interrupción; una delección o un intrón parten un alineamiento en varios bloques.

Calcular la ecuación (7.19) de forma directa cuesta $O(\sum_a |a|)$: unos 10^{11} incrementos para un genoma a 30×. Pedersen y Quinlan (2018) observaron en `mosdepth`

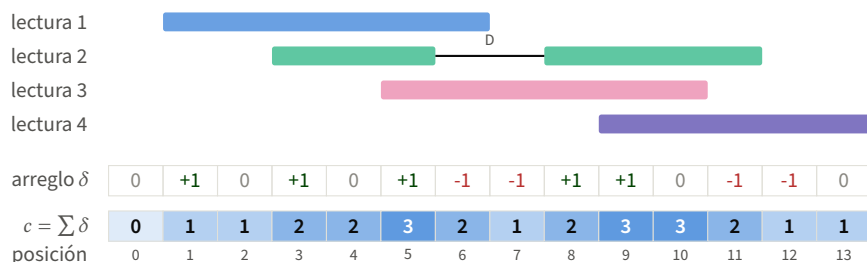


Figura 7.11. El algoritmo de `mosdepth` (ecuación (7.20)) sobre 14 posiciones. Cada bloque alineado suma 1 en su inicio y resta 1 en su final (posición siguiente a la última cubierta). La lectura 2 contiene una deleción (D) y aporta dos bloques. La suma acumulada del arreglo δ reproduce la profundidad de cada posición, coloreada de claro a oscuro. Con N bloques y un cromosoma de longitud G , el coste es $O(N + G)$, sin importar cuántas lecturas se apilen en cada base.

que basta con anotar dónde *empieza* y dónde *termina* cada bloque en un arreglo de diferencias δ del tamaño del cromosoma,

$$\delta(s) += 1, \quad \delta(e) -= 1 \quad \text{para cada bloque } [s, e), \quad c(x) = \sum_{y \leq x} \delta(y), \quad (7.20)$$

y reconstruir la profundidad con una suma acumulada. El coste pasa a ser proporcional al número de bloques más la longitud del cromosoma, y no a la profundidad (figura 7.11). El precio es memoria: un entero de 32 bits por base, cerca de 1 GB para el cromosoma 1 humano. `mosdepth` evita además contar dos veces la región donde los dos extremos de un par se solapan, que no aporta información independiente. Otras herramientas cubren usos distintos: `samtools depth`, `bedtools genomecov` (Quinlan & Hall, 2010) y `bamCoverage` de `deepTools`, que produce pistas normalizadas en formato `bigWig` para comparar muestras en un navegador (Ramírez et al., 2016).

`profundidad.py`

```
import numpy as np
import pysam

def profundidad(bam, crom, mapq=20):
    """Profundidad por base con el arreglo de diferencias."""
    with pysam.AlignmentFile(bam) as f:
        G = f.get_reference_length(crom)
        d = np.zeros(G + 1, dtype=np.int32)
```

```

for a in f.fetch(crom):
    if (a.is_unmapped or a.is_secondary or a.is_supplementary
        or a.is_duplicate or a.mapping_quality < mapq):
        continue
    for s, e in a.get_blocks():           # bloques M/=X
        d[s] += 1
        d[e] -= 1
return np.cumsum(d[:-1])

```

Cobertura con mosdepth

```

# profundidad media en ventanas de 500 pb, solo MAPQ >= 20
mosdepth -t 4 -n --by 500 -Q 20 m1 m1.bam
zcat m1.regions.bed.gz | head -3      # crom inicio fin media
cat m1.mosdepth.summary.txt          # media por cromosoma

```

7.3.3 Cobertura y número de copias

Si las lecturas se muestrean uniformemente, la profundidad esperada en una región es proporcional a su número de copias. Para un genoma diploide,

$$\mathbb{E}[c(x)] = \bar{c} \frac{\text{CN}(x)}{2} b(x), \quad (7.21)$$

y el estadístico natural para comparar una ventana W con el resto del genoma es el logaritmo del cociente

$$\ell_W = \log_2 \frac{\bar{c}_W}{\text{mediana}_V \bar{c}_V}. \quad (7.22)$$

DONDE

$\bar{c}$	Cobertura media del genoma (p. ej., 30×).
$\text{CN}(x)$	Número de copias de la región en la muestra: 2 normal, 1 deleción heterocigota, o homocigota, 3 duplicación heterocigota.
$b(x)$	Sesgo local multiplicativo (contenido de GC, mapeabilidad), idealmente 1.
$\bar{c}_W$	Profundidad media en la ventana W .
ℓ_W	Razón logarítmica: 0 normal, -1 deleción heterocigota, $\log_2 1,5 = 0,58$ duplicación, $-\infty$ deleción homocigota.

La figura 7.12 muestra estos patrones en un perfil simulado. Obsérvese la trampa de la repetición: con todas las lecturas, su cobertura es normal (unos 35); con el

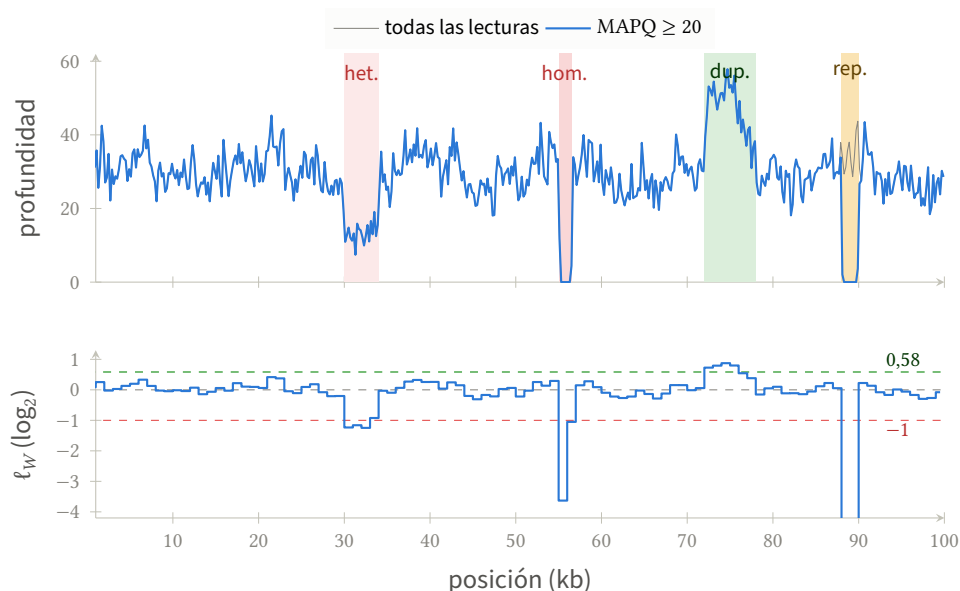


Figura 7.12. Perfil de cobertura simulado de 100 kb a 30× con lecturas de 150 pb. **Arriba:** profundidad media en bloques de 200 pb, con todas las lecturas (gris) y solo con $\text{MAPQ} \geq 20$ (azul). Se aprecian una deleción heterocigota de 4 kb (la profundidad cae a la mitad), una homocigota de 1,5 kb (cae a cero), una duplicación de 6 kb (sube un 50 %) y una repetición de 2 kb cuya otra copia está en el genoma: las lecturas se mapean pero con $\text{MAPQ} = 0$, así que la cobertura filtrada se desploma aunque no falte ADN. El sesgo ondulante imita el efecto del contenido de GC. **Abajo:** razón logarítmica en ventanas de 1 kb (ecuación (7.22)); las ventanas de la deleción heterocigota promedian $-1,14$ y las de la duplicación $0,69$, cerca de los valores teóricos (líneas discontinuas).

filtro de $\text{MAPQ} \geq 20$, cae a 0,6 y se confundiría con una delección homocigota. Un análisis de cobertura siempre debe acompañarse de una máscara de *mapeabilidad* que diga qué regiones del genoma pueden recibir lecturas únicas. CNVnator, uno de los métodos clásicos de profundidad de lectura, corrige el sesgo de GC, segmenta el perfil con un algoritmo de *mean-shift* y alcanza, con cobertura alta, una resolución de los puntos de corte de menos de 200 pb en el 90 % de los casos (Abyzov et al., 2011). Sus autores subrayan que la profundidad es complementaria a las lecturas discordantes y partidas: cada señal detecta variantes que las otras no ven.

7.3.4 ¿Qué delección es detectable?

Hagamos la cuenta que decide si una delección se ve o se pierde en el ruido. En una ventana de W bases con cobertura $\bar{c}$ y lecturas de longitud L , el número de lecturas que empiezan en la ventana es aproximadamente de Poisson con media $\lambda = \bar{c}W/L$, y su desviación típica es $\sqrt{\lambda}$. Una delección heterocigota reduce la media a $\lambda/2$; la diferencia, medida en desviaciones típicas de la hipótesis nula, es

$$z_{\text{het}} = \frac{\lambda - \lambda/2}{\sqrt{\lambda}} = \frac{\sqrt{\lambda}}{2}, \quad z_{\text{hom}} = \frac{\lambda}{\sqrt{\lambda}} = \sqrt{\lambda}. \quad (7.23)$$

Exigiendo $z \geq z^*$ y despejando W ,

$$W_{\min}^{\text{het}} = \frac{4 z^{*2} L}{\bar{c}}, \quad W_{\min}^{\text{hom}} = \frac{z^{*2} L}{\bar{c}}. \quad (7.24)$$

DONDE

λ	Número esperado de lecturas que empiezan en la ventana: $\bar{c}W/L$.
z^*	Umbral de significación en desviaciones típicas; con millones de ventanas en el genoma, $z^* \approx 5$ controla los falsos positivos.
$W_{\min}$	Tamaño mínimo de una delección detectable por profundidad.

Ejemplo 7.4. (Diseño para detectar delecciones)

Con $\bar{c} = 30$, $L = 150$ y $z^* = 5$, la ecuación (7.24) da $W_{\min}^{\text{het}} = 4 \cdot 25 \cdot 150/30 = 500$ pb y $W_{\min}^{\text{hom}} = 125$ pb. A $10\times$, los valores se triplican: 1500 y 375 pb; a $60\times$, se reducen a la mitad: 250 y 62 pb. Una duplicación heterocigota, que cambia la media de λ a $1,5\lambda$, produce la misma diferencia absoluta que una delección heterocigota y requiere, por tanto, ventanas similares. Por debajo de unos

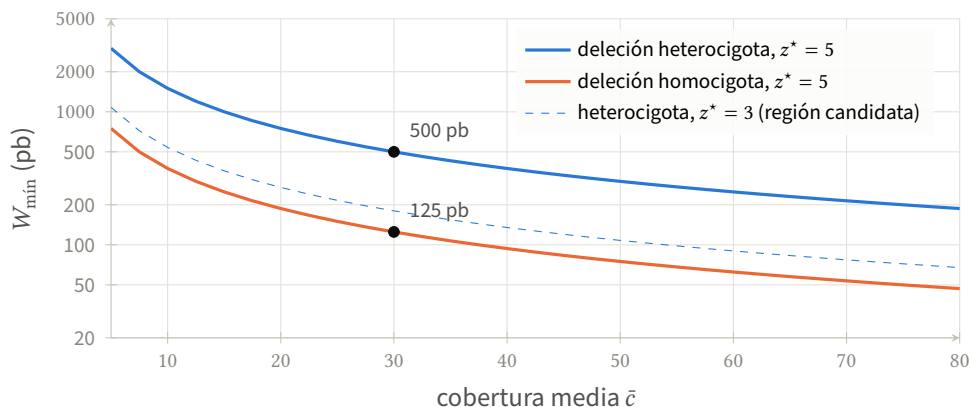


Figura 7.13. Tamaño mínimo de una deleción detectable solo por profundidad, según la ecuación (7.24), para lecturas de 150 pb. Duplicar la cobertura reduce a la mitad el tamaño detectable; pasar de una búsqueda en todo el genoma ($z^* = 5$) a la comprobación de una región candidata ($z^* = 3$) lo reduce casi a un tercio. El modelo es optimista: supone muestreo de Poisson perfecto, y el sesgo residual tras la corrección de GC aumenta la varianza.

cientos de bases, la profundidad no basta y hay que recurrir a lecturas partidas y a pares discordantes (figura 7.8).

El techo de la sobredispersión

Si el recuento de una ventana no es de Poisson sino binomial negativo, con varianza $\lambda + \lambda^2/r$ como en el ??, la ecuación (7.23) se convierte en

$$z_{\text{het}} = \frac{\lambda/2}{\sqrt{\lambda + \lambda^2/r}} \xrightarrow{\lambda \rightarrow \infty} \frac{\sqrt{r}}{2}.$$

La señal ya no crece indefinidamente con la ventana ni con la cobertura: tiene un techo fijado por la sobredispersión. Con $r = 50$, z_{het} nunca supera 3,5, por mucho que se secuencie. Esta es la razón matemática por la que los métodos de profundidad dedican tanto esfuerzo a corregir los sesgos de GC y de mapeabilidad: reducir la varianza sistemática rinde más que añadir lecturas.

La cobertura es una medida del número de copias, pero solo después de filtrar duplicados, controlar la calidad de mapeo y corregir los sesgos; una caída de cobertura puede ser una delección o una repetición.

☰ En resumen

- Mapear es encontrar el origen de cada lectura y cuantificar la confianza en él. La programación dinámica exhaustiva es inviable a escala genómica; los mapeadores filtran con índices y reservan el alineamiento para unas pocas regiones candidatas.
- El arreglo de sufijos agrupa las apariciones de un patrón en un intervalo. La BWT es la columna de caracteres precedentes de ese arreglo; el *LF-mapping* ($LF(r) = C[L_r] + \text{Occ}(L_r, r)$) la hace reversible y la búsqueda hacia atrás encuentra un patrón con dos consultas por carácter.
- Los puntos de control de Occ y el muestreo del arreglo de sufijos reducen el índice FM del genoma humano a 1–2 GB, a cambio de más pasos de *LF-mapping* por posición localizada.
- BWA-MEM siembra con SMEM sobre el índice FM; minimap2 con minimizadores, cuya densidad es $\approx 2/(w + 1)$. Ambos encadenan anclas colineales y extienden con programación dinámica en banda.
- El MAPQ es la probabilidad, en escala Phred, de que la posición asignada sea errónea; es bajo en repeticiones aunque el alineamiento sea perfecto, y no es comparable entre programas.
- SAM/BAM guarda cada alineamiento con su FLAG, su CIGAR y la información de la pareja; *samtools* ordena, marca duplicados e indexa. Los pares discordantes y las lecturas partidas señalan variantes estructurales.
- La profundidad por base se calcula en tiempo lineal con un arreglo de diferencias; su razón logarítmica revela delecciones $(-1, -\infty)$ y duplicaciones $(0, 58)$, con un tamaño mínimo detectable inversamente proporcional a la cobertura.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 7.1 ●●● Construya a mano el arreglo de sufijos y la BWT de GATTACA\$. Compruebe con `indice_fm` su resultado y verifique que las letras iguales de L y F conservan su orden relativo.
- 7.2 ●●● Usando la tabla 7.1, ejecute la búsqueda hacia atrás de TAC y de CAT en TACATACAG\$. Dé los intervalos intermedios y las posiciones finales.
- 7.3 ●●● Decodifique los FLAG 83, 147, 163 y 2064. ¿Cuáles pertenecen a pares concordantes? ¿Cuál es un alineamiento suplementario y qué sugiere sobre la lectura?
- 7.4 ●●● Con la ecuación (7.9), calcule la memoria del índice FM para el genoma de trigo ($1,6 \times 10^{10}$ bases) con $d = 128$ y $s = 32$. ¿Cuántos bits necesita cada entero? Discuta qué intervalo s elegiría para que el índice quepa en 32 GB.
- 7.5 ●●● Implemente la reconstrucción de T a partir de L con el *LF-mapping* y compruebe que funciona con una secuencia aleatoria de 10 000 bases.
- 7.6 ●●● Simule secuencias aleatorias y mida la densidad de los minimizadores con orden alfabético y con un *hash* aleatorio para $w \in \{5, 10, 20\}$ y $k = 15$. Compare con la ecuación (7.11). ¿Por qué el orden alfabético es peor en secuencias con muchas A?
- 7.7 ●●● Una lectura tiene tres ubicaciones: una con una discordancia de calidad 30, y dos con dos discordancias de calidades 30 y 30. Calcule el MAPQ con la ecuación (7.16). ¿Cómo cambia si la calidad de la única discordancia de la primera es 10?
- 7.8 ●●● Descargue un BAM público de una muestra humana (por ejemplo, del Proyecto 1000 Genomas), calcule con `mosdepth` la profundidad en ventanas de 1 kb de un cromosoma y busque tramos con $\ell_W < -0,7$. Contraste sus candidatos con las lecturas discordantes vistas en IGV.
- 7.9 ●●● Demuestre la aproximación $2/(w+1)$ de la ecuación (7.11) con detalle, suponiendo que los valores de *hash* son variables continuas independientes. ¿Qué ocurre con la densidad en una región de baja complejidad como **A**T**A**T**A**
TA**T**A**T**?

Referencias del capítulo

- Abyzov, A., Urban, A. E., Snyder, M., & Gerstein, M. (2011). CNVnator: An approach to discover, genotype, and characterize typical and atypical CNVs from family and population genome sequencing. *Genome Research*, 21(6), 974-984. <https://doi.org/10.1101/gr.114876.110>
- Adjeroh, D., Bell, T., & Mukherjee, A. (2008). *The Burrows-Wheeler Transform: Data Compression, Suffix Arrays, and Pattern Matching*. Springer. <https://doi.org/10.1007/978-0-387-78909-5>
- Danecek, P., Bonfield, J. K., Liddle, J., Marshall, J., Ohan, V., Pollard, M. O., Whitwham, A., Keane, T., McCarthy, S. A., Davies, R. M., & Li, H. (2021). Twelve years of SAMtools and BCFtools. *GigaScience*, 10(2), Artículo giab008. <https://doi.org/10.1093/gigascience/giab008>
- Faust, G. G., & Hall, I. M. (2014). SAMBLASTER: fast duplicate marking and structural variant read extraction. *Bioinformatics*, 30(17), 2503-2505. <https://doi.org/10.1093/bioinformatics/btu314>
- Ferragina, P., & Manzini, G. (2005). Indexing compressed text. *Journal of the ACM*, 52(4), 552-581. <https://doi.org/10.1145/1082036.1082039>
- Gusfield, D. (1997). *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511574931>
- Langmead, B., & Salzberg, S. L. (2012). Fast gapped-read alignment with Bowtie 2. *Nature Methods*, 9(4), 357-359. <https://doi.org/10.1038/nmeth.1923>
- Langmead, B., Trapnell, C., Pop, M., & Salzberg, S. L. (2009). Ultrafast and memory-efficient alignment of short DNA sequences to the human genome. *Genome Biology*, 10(3), Artículo R25. <https://doi.org/10.1186/gb-2009-10-3-r25>
- Li, H. (2013). *Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM*. arXiv: 1303.3997 [q-bio.GN]. <https://doi.org/10.48550/arXiv.1303.3997>
- Li, H. (2018). Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics*, 34(18), 3094-3100. <https://doi.org/10.1093/bioinformatics/bty191>
- Li, H., & Durbin, R. (2009). Fast and accurate short read alignment with Burrows-Wheeler transform. *Bioinformatics*, 25(14), 1754-1760. <https://doi.org/10.1093/bioinformatics/btp324>
- Li, H., & Durbin, R. (2010). Fast and accurate long-read alignment with Burrows-Wheeler transform. *Bioinformatics*, 26(5), 589-595. <https://doi.org/10.1093/bioinformatics/btp698>
- Li, H., Handsaker, B., Wysoker, A., Fennell, T., Ruan, J., Homer, N., Marth, G., Abecasis, G., Durbin, R., & 1000 Genome Project Data Processing Subgroup. (2009). The Sequence Alignment/Map format and SAMtools. *Bioinformatics*, 25(16), 2078-2079. <https://doi.org/10.1093/bioinformatics/btp352>

- Li, H., & Homer, N. (2010). A survey of sequence alignment algorithms for next-generation sequencing. *Briefings in Bioinformatics*, 11(5), 473-483. <https://doi.org/10.1093/bib/bbq015>
- Li, H., Ruan, J., & Durbin, R. (2008). Mapping short DNA sequencing reads and calling variants using mapping quality scores. *Genome Research*, 18(11), 1851-1858. <https://doi.org/10.1101/gr.078212.108>
- Mäkinen, V., Belazzougui, D., Cunial, F., & Tomescu, A. I. (2015). *Genome-Scale Algorithm Design: Biological Sequence Analysis in the Era of High-Throughput Sequencing*. Cambridge University Press. <https://doi.org/10.1017/CBO9781139940023>
- Manber, U., & Myers, G. (1993). Suffix arrays: A new method for on-line string searches. *SIAM Journal on Computing*, 22(5), 935-948. <https://doi.org/10.1137/0222058>
- Nurk, S., Koren, S., Rhie, A., Rautiainen, M., et al. (2022). The complete sequence of a human genome. *Science*, 376(6588), 44-53. <https://doi.org/10.1126/science.abj6987>
- Pedersen, B. S., & Quinlan, A. R. (2018). Mosdepth: quick coverage calculation for genomes and exomes. *Bioinformatics*, 34(5), 867-868. <https://doi.org/10.1093/bioinformatics/btx699>
- Quinlan, A. R., & Hall, I. M. (2010). BEDTools: a flexible suite of utilities for comparing genomic features. *Bioinformatics*, 26(6), 841-842. <https://doi.org/10.1093/bioinformatics/btq033>
- Ramírez, F., Ryan, D. P., Grüning, B., Bhardwaj, V., Kilpert, F., Richter, A. S., Heyne, S., Dündar, F., & Manke, T. (2016). deepTools2: a next generation web server for deep-sequencing data analysis. *Nucleic Acids Research*, 44(W1), W160-W165. <https://doi.org/10.1093/nar/gkw257>
- Roberts, M., Hayes, W., Hunt, B. R., Mount, S. M., & Yorke, J. A. (2004). Reducing storage requirements for biological sequence comparison. *Bioinformatics*, 20(18), 3363-3369. <https://doi.org/10.1093/bioinformatics/bth408>
- Robinson, J. T., Thorvaldsdóttir, H., Winckler, W., Guttman, M., Lander, E. S., Getz, G., & Mesirov, J. P. (2011). Integrative genomics viewer. *Nature Biotechnology*, 29(1), 24-26. <https://doi.org/10.1038/nbt.1754>
- SAM/BAM Format Specification Working Group. (2024). *Sequence Alignment/Map Format Specification*. GA4GH / hts-specs. <https://samtools.github.io/hts-specs/SAMv1.pdf>
- Schleimer, S., Wilkerson, D. S., & Aiken, A. (2003). Winnowing: Local algorithms for document fingerprinting. *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*, 76-85. <https://doi.org/10.1145/872757.872770>
- Schneider, V. A., Graves-Lindsay, T., Howe, K., Bouk, N., Chen, H.-C., Kitts, P. A., Murphy, T. D., Pruitt, K. D., Thibaud-Nissen, F., Albracht, D., et al. (2017). Evaluation of GRCh38 and de novo haploid genome assemblies demonstrates the enduring quality of the reference assembly. *Genome Research*, 27(5), 849-864. <https://doi.org/10.1101/gr.213611>

- Thorvaldsdóttir, H., Robinson, J. T., & Mesirov, J. P. (2013). Integrative Genomics Viewer (IGV): high-performance genomics data visualization and exploration. *Briefings in Bioinformatics*, 14(2), 178-192. <https://doi.org/10.1093/bib/bbs017>
- Treangen, T. J., & Salzberg, S. L. (2012). Repetitive DNA and next-generation sequencing: computational challenges and solutions. *Nature Reviews Genetics*, 13(1), 36-46. <https://doi.org/10.1038/nrg3117>